



Homework #1

TurtleSim Digit Drawing

Capstone Design [202601-ICE/CSE4020]
Individual Assignment

1. Overview

In this homework, you will write a single Python ROS node that launches four turtles in a `turtlesim_node` window and commands each turtle to draw one digit. The four digits must correspond to the **last four digits of your student ID**, and all four turtles must draw their respective digits simultaneously (in parallel). By the end of the task, the turtlesim window should display a clear, readable four-digit number.

This assignment is designed to strengthen your understanding of core ROS concepts such as topics, services, publishers, and multi-entity coordination — all of which are foundational to working with TurtleBot3 in later modules.

2. ROS Concepts Refresher

Before diving into the task, make sure you are comfortable with the following ROS concepts. These are the building blocks you will need for this homework.

2.1 Nodes

A node is an executable process that communicates with other nodes through the ROS framework. In this homework, `turtlesim_node` is one node (the simulator), and your Python script will be another node. You initialize your node with `rospy.init_node('your_node_name')`.

2.2 Topics and Publishers

Topics are named channels through which nodes exchange messages. A publisher sends messages to a topic, and any node that subscribes to that topic will receive them. For turtlesim, each turtle has its own velocity topic. For example, `/turtle1/cmd_vel` accepts `geometry_msgs/Twist` messages that control turtle1's linear and angular velocity. When you spawn additional turtles, they get their own topics: `/turtle2/cmd_vel`, `/turtle3/cmd_vel`, and so on.

2.3 Services

Services follow a request-response model, unlike the one-way nature of topics. You will need several turtlesim services for this homework. The `/spawn` service creates a new turtle at a given

position and orientation. The `/turtleX/teleport_absolute` service moves a turtle instantly to specific coordinates without drawing. The `/turtleX/set_pen` service controls the pen color, width, and whether it is on or off. You call services using `rospy.ServiceProxy`.

2.4 The Twist Message

The `geometry_msgs/Twist` message has two vector fields: `linear` (with x, y, z components) and `angular` (with x, y, z components). For a 2D turtle, only `linear.x` (forward/backward speed) and `angular.z` (turning rate) are meaningful. Setting `linear.x = 1.0` moves the turtle forward at 1 unit/second, and setting `angular.z = 1.57` rotates it counterclockwise at roughly 90°/second.

2.5 Concurrent Execution

Since all four turtles must draw at the same time, you will need to explore concurrent execution in Python. The `threading` module is one straightforward approach: you can define a drawing function for one turtle and then launch four threads, one per turtle. Alternatively, you can publish to all four velocity topics in a single loop using timed sequences — the key is that the turtles must visibly move simultaneously, not one after another.

3. Task Description

Your task, step by step, is as follows.

1. **Launch the simulator.** Start `turtlesim_node` in a separate terminal before running your script. Your node should connect to the already-running simulator.
2. **Spawn additional turtles.** The simulator starts with one turtle (`turtle1`). Use the `/spawn` service to create three more, giving you a total of four turtles.
3. **Position the turtles.** Teleport each turtle to a starting location so the four digits will appear side by side without overlapping. Think about dividing the canvas into four vertical zones. Remember that the turtlesim canvas is approximately 11.1×11.1 units.
4. **Design digit shapes.** Plan the movement sequences that trace each digit (0–9). Each digit should be drawn using straight lines and turns. You do not need pixel-perfect rendering — the digits just need to be clearly recognizable. Consider using the pen service to lift the pen when repositioning within a digit.
5. **Draw all four digits in parallel.** This is the core challenge. All four turtles must be actively drawing at the same time. A sequential approach (draw digit 1, then digit 2, then digit 3, then digit 4) will **not** receive full marks.
6. **Display your student ID digits.** The four digits drawn must be the last four digits of your own student ID. For example, if your ID is U2010164, the turtles should draw 0, 1, 6, and 4 from left to right.

Hints

- Use `rospy`. `Rate` and a `while` loop to publish `Twist` messages at a consistent frequency (e.g., 10 Hz).
- A digit drawing function that takes a turtle name and a digit as arguments will make your code clean and reusable.
- Think of each digit as a sequence of actions: move forward, turn, lift pen, teleport, lower pen, etc.
- You can explore the available services manually first by running: `rosservice list` and `rosservice info /spawn`.
- Test with simple shapes first (e.g., draw a square) before attempting complex digit paths.

4. Grading Rubric

Your submission will be evaluated according to the following criteria. The total score is 100 points.

Criterion	Points	Details
Correct digit rendering font size matters	25	All four digits are clearly recognizable and match the last four digits of the student ID.
Parallel execution	30	All four turtles draw simultaneously, verified via the screen recording.
Quality of written digits font style matters	20	Results are of a sufficient quality - written digits don't consist of straight lines only, and the student tries to implement curvatures.
Proper use of ROS services	10	Correct use of <code>spawn</code> , <code>teleport</code> , <code>set_pen</code> , and other services in the code.
Code quality and structure	10	Clean, readable code with meaningful names and comments explaining key logic.
Screen recording clearance	5	The recording clearly shows the full drawing process from start to finish.

5. Submission Instructions

You must submit the following two items.

1. **Python source code** — Your complete ROS node as a single `.py` file. The file should be self-contained and runnable. Name it using the format `hw1_STUDENTID.py` (for example, `hw1_U2010164.py`).
2. **Screen recording** — A video showing the turtlesim window from launch through the completion of all four digits. The recording should clearly demonstrate that all four turtles are moving simultaneously. Accepted formats: MP4, AVI, or MKV. Keep the file under 100 MB.

Upload both files to the **GitHub repository of IUT** designated for this course. Further instructions on repository access and submission structure will be announced on our **Discord server**. Make sure to follow the announced structure carefully. *Late submissions will be penalized!*

6. Important Notes

1. This is an **individual assignment**. You must write your own code independently. Sharing code with classmates is considered academic dishonesty.
2. Make sure your environment is set up correctly: `Ubuntu 20.04` with `ROS Noetic` installed and sourced in your terminal.
3. Before writing code, spend time exploring turtlesim manually using the command line. Commands like `rostopic list`, `rosservice list`, and `rostopic pub` will help you understand the interface.
4. If you encounter issues, check the **ROS Wiki turtlesim page** and the official ROS Noetic tutorials. These are your primary reference materials.
5. Start early. Designing the movement paths and coordinating four threads takes more time than you might expect.

Good luck! If you have any questions, reach out to the Education Assistant of the course.